

Gestão Clínica Acadêmica

Documento Oficial de Arquitetura, Engenharia e Integração (Departamento de TI - UMJ)

1 Arquitetura e Padrões de Projeto

A aplicação foi projetada utilizando uma arquitetura baseada em microsserviços providos por containers Docker, separando responsabilidades para garantir isolamento, resiliência e facilidade de escala (Scale-out).

Stack Tecnológica e Racional

Python 3.11

Flask 3.0

PostgreSQL 16

Redis

Celery

Gunicorn + Gevent

Docker Engine

A escolha do **Python + Flask** proporciona uma base de código modular e leve, adequada à lógica de regras acadêmicas dinâmicas. Empregamos o padrão MVC (Model-View-Controller) através do uso de Blueprints do Flask, o que segmenta a aplicação em módulos lógicos isolados (ex: `/auth`, `/patients`, `/clinical`, `/admin`).

O servidor WSGI nativo é o **Gunicorn**, configurado com workers assíncronos (**Gevent**). Esta configuração permite que o sistema manipule milhares de conexões HTTP abertas simultaneamente sem bloqueio de I/O, vital para ambientes acadêmicos onde o uso simultâneo do sistema ocorre em picos durante as clínicas.

2 Topologia de Containers (Docker)

A orquestração local é feita via `docker-compose`, garantindo paridade total entre ambientes de desenvolvimento, homologação e produção.

Serviço	Tecnologia	Função no Ecossistema
Web (App)	Python/Flask + Gunicorn	Exposição dos endpoints HTTP, renderização de templates Jinja2 e controle do fluxo de negócios.
PostgreSQL	PostgreSQL 16	Repositório relacional transacional (ACID). Isolado da rede externa (acesso apenas via bridge virtual).
Redis	Redis In-Memory	Broker de mensagens para o Celery, controle de sessões e filas de cache.
Celery Worker	Python/Celery	Processamento assíncrono de rotinas pesadas (ex: processamento de imagens e renderização de PDFs).

3 Modelo de Dados e Armazenamento

Banco de Dados Relacional (PostgreSQL)

O schema do banco de dados foi construído focado em integridade referencial. Utilizamos o ORM SQLAlchemy para mapeamento. Tabelas críticas incluem:

- **Users:** Controle de acesso com hierarquia (Administrador, Professor e Aluno).
- **Patients:** Dados demográficos, anamneses e histórico médico (cruzamento direto com evolução).
- **Treatments & Evolutions:** Registro de cada intervenção. Há relacionamentos estritos, garantindo que evoluções só fiquem como "concluídas" após assinatura digital dupla (Aluno + Professor).

Gerenciamento de Arquivos e Mídia (Uploads)

O armazenamento de imagens de exames e radiografias utiliza um pipeline de otimização próprio. O servidor intercepta o upload, e via biblioteca **Pillow**, reduz as dimensões para limites seguros (Max 1920px) e qualidade 85% para JPEG, eliminando metadados EXIF por segurança. O arquivo otimizado é salvo no volume Docker mapeado em `/app/uploads`.

Controle de Acesso (RBAC)

A aplicação utiliza Role-Based Access Control rígido. Rotas sensíveis (como validação de evoluções ou exclusão de registros) possuem decoradores no nível do controlador (Flask `@login_required` e verificações de cargo). Senhas são armazenadas de forma segura utilizando criptografia de hash unidirecional PBKDF2 (padrão do módulo Werkzeug Security).

Assinatura Eletrônica e Rastreabilidade

Para conformidade com práticas de prontuário eletrônico no ensino odontológico:

- **Paciente:** Captura de assinatura por biometria comportamental via interface Canvas (HTML5), convertida para Base64 e persistida com timestamp direto no banco de dados.
- **Professor:** Validação de etapas do plano de tratamento através de confirmação de credencial ativa, garantindo a autoria e rastreabilidade da aprovação.

Isolamento de Rede: O banco de dados e o Redis não possuem portas expostas para o host externo. Toda a comunicação ocorre exclusivamente através da rede virtual interna (bridge) criada pelo Docker.

Considerações para a TI (Deploy e Escalabilidade)

A arquitetura entrega a aplicação de forma *Stateless* (livre de estado na camada de processamento web), o que significa que o ambiente está totalmente preparado para escalabilidade horizontal:

- O diretório de imagens (`/app/uploads`) e o diretório de dados do Postgres (`pgdata`) são montados em volumes persistentes do Docker. O ciclo de vida do container não interfere na persistência da informação física.
- Os relatórios PDF não consomem armazenamento definitivo. Eles são gerados em tempo real (On-the-Fly) utilizando WeasyPrint e transmitidos via streaming HTTP, sem deixar arquivos temporários residuais no servidor.
- Atualizações (CI/CD) consistem unicamente em um `git pull` seguido por `docker compose build --no-cache web` e `docker compose up -d` , resultando em Zero Downtime percebido para o usuário final.